

TEST008 - Error Handling Middleware Testing

This document records the testing evidence and result for ticket ****TEST008****.

Ticket intent

- Trigger API errors.
- Check response format.
- Confirm consistent error responses.
- Confirm proper status codes.

Scope

Validated backend error handling behavior implemented in:

- `database/logging\_system/request\_middleware.py`
- `database/logging\_system/exception\_handler.py`

Automated tests added in:

- `test/test\_t1008\_error\_handling\_middleware.py`

Test cases implemented

1. ****Success path includes request traceability****

- Call `GET /ok`.
- Expect HTTP `200`.
- Expect body: `{"ok": true}`.

- Expect `X-Request-ID` response header is present.

2. ****Error path returns consistent format and status****

- Call `GET /explode` (forced runtime exception).
- Expect HTTP `500`.
- Expect body includes:
 - `message` = `"Internal server error"`
 - `request\_id` (non-empty)
- Expect `X-Request-ID` header equals body `request\_id`.

Fix applied during testing

While executing TEST008, one assertion failed because error responses did not include the `X-Request-ID` header.

To align success and error behavior, `global\_exception\_handler` was updated to set:

```
- `headers={"X-Request-ID": request_id}`
```

in the returned `JSONResponse`.

Execution evidence

Command run from repository root:

```
` `` bash
python -m pytest test/test_t1008_error_handling_middleware.py -q
` ``
```

Observed result:

```text

.. [100%]

2 passed in 0.30s

```

Final result

TEST008 acceptance criteria are satisfied by automated tests:

- API errors are triggered and validated.
- Error response format is consistent.
- Proper status codes are returned (`200` success, `500` unhandled error).
- `request\_id` tracing is consistent in both payload and response header.